Part A: Requirements Engineering Theory

1. Requirements Candidates

In the early stages of software development, a **requirement candidate** is a potential feature, constraint, or capability identified during the initial discovery phase. Think of them as "raw data" or "nominations" for what the system should eventually do.

Why they aren't "Final" immediately:

It is a common mistake to treat every initial request as a concrete requirement. They must undergo a rigorous process of analysis and negotiation for several reasons:

- **Conflict and Overlap:** Different stakeholders often provide contradictory requests. One user might want "maximum security" (which slows the system), while another wants "instant access" (which bypasses security).
- **Feasibility Issues:** A candidate might be technically impossible to build within the current budget or timeframe.
- **Ambiguity:** Initial elicitations are often vague. A request for a "fast interface" isn't a requirement until it is quantified (e.g., "page load under 2 seconds").
- **Scope Creep:** Without filtering candidates, the project scope can balloon, leading to missed deadlines and "bloated" software that is hard to use.

2. The Role of Stakeholders

Stakeholders are individuals, groups, or organizations that have a "stake" in the project—meaning they can affect or be affected by the system's outcome.

In **elicitation**, their role is foundational:

- **Source of Truth:** They provide the domain knowledge and business logic that developers lack.
- **Goal Setting:** They define the "why" behind the project (the business objectives).
- **Validation:** They review the requirement candidates to ensure the technical team hasn't misinterpreted their needs.

Risks of Missing Key Stakeholders

If a critical stakeholder (like an end-user or a regulatory compliance officer) is overlooked, the project faces significant risks:

1. **Requirement Gaps (Omissions):** You may build a technically perfect system that is missing a vital legal feature (like GDPR compliance) or a functional step that a specific department needs to do their job.
2. **User Rejection:** If the people actually using the software daily aren't consulted, the final product may be "clunky" or counter-intuitive to their workflow, leading to low adoption rates and a wasted investment.